

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naïve Bayes and KNN ¹
Student Name	Danusu K
Register Number	3122247001013
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	August 2, 2026

1 Objective

Build and evaluate binary classifiers for email spam/ham detection using Naïve Bayes and K-Nearest Neighbors (KNN) algorithms. Compare Gaussian, Multinomial, and Bernoulli Naïve Bayes variants, and study the effect of varying the neighborhood parameter k . Perform spatial index structure comparisons (KDTree vs. BallTree), execute hyperparameter optimization (GridSearchCV vs. RandomizedSearchCV), and analyze theoretical and practical time complexities.

2 Problem Statement

Email spam poses significant security threats and performance issues. The problem is to develop a machine learning pipeline to classify emails as Ham (legitimate, class 0) or Spam (unsolicited bulk, class 1). The input is a feature vector of word and character occurrences from the Spambase dataset, and the output is the binary prediction. The objective is to design classifiers that achieve high predictive performance (accuracy, precision, recall, and F1-score) while minimizing prediction times.

3 Dataset Description

Dataset Name	Spambase Dataset
Dataset Source	UCI Machine Learning Repository
Number of Samples	4601
Number of Features	57
Number of Classes	2
Missing Values	0
Train-Test Split	80% Train, 20% Test

4 Exploratory Data Analysis

Exploratory Data Analysis was performed to evaluate the structure, distribution, and relationship patterns within the Spambase dataset:

- **Dataset Overview:** Inspecting data shapes, columns, and target fields.

¹GitHub Repository: <https://github.com/Danush-k/Machine-Learning>

- **Statistical Summary:** Computing features' ranges, mean values, standard deviations, and skewness.
- **Missing Value Analysis:** Scanning the dataset to ensure completeness.
- **Class Distribution:** Evaluating target variable balance.
- **Correlation Matrix:** Analyzing linear dependency between the predictor columns and target class.
- **Feature Distribution:** Inspecting spread and density shape of individual key features.



Figure 1: Spam vs. Ham Class Count Distribution

Inference: The dataset contains 4,601 instances with a target distribution of approximately 60.6% ham (2,788 instances) and 39.4% spam (1,813 instances). The class distribution shows that while there is a slight imbalance, it is not severe enough to require special resampling or class-weight adjustment. Performance evaluation can rely safely on conventional accuracy, precision, and F1 metrics.



Figure 2: Correlation Heatmap of Top Features with Class Target

Inference: The heatmap displays the Pearson correlation values for the 12 features most strongly associated with the target class. Features such as `word_freq_your`, `word_freq_000`, and `char_freq_$` exhibit strong positive correlations with the spam class, indicating they occur frequently in spam emails. In contrast, personal identifier frequencies show weak negative correlations, making them indicators of ham emails.

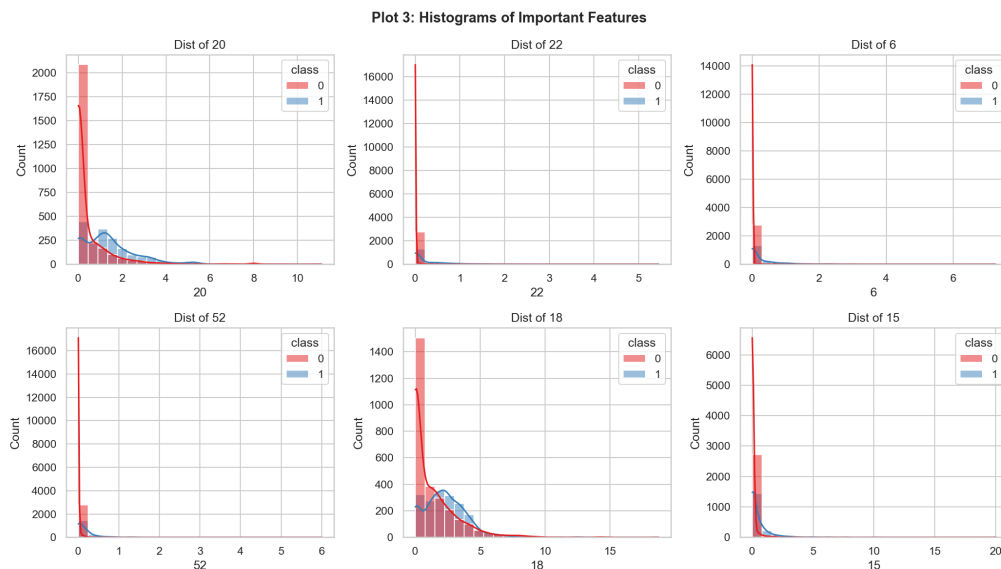


Figure 3: Histograms of Most Important Predictor Features

Inference: The histograms compare the frequency distributions of the top 6 predictor features across the two classes. Most frequency values are close to zero for ham emails, demonstrating the sparsity of commercial keywords in regular text. Spam emails exhibit

significantly wider distributions with heavier tails, suggesting that these terms occur repeatedly when a message is unsolicited.

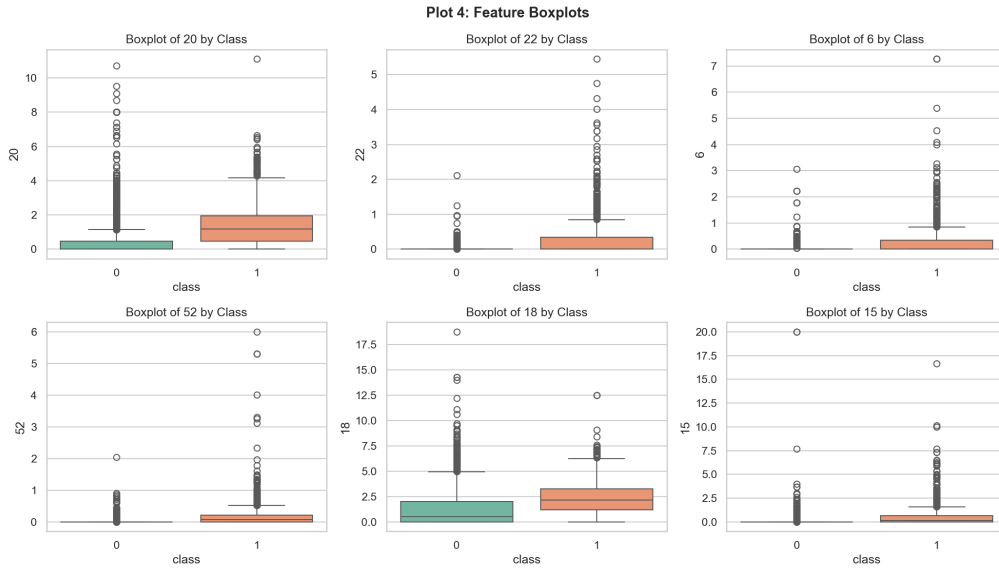


Figure 4: Boxplots of Key Features Split by Spam and Ham Class

Inference: The boxplots represent the median, quartiles, and outlier thresholds for the top 6 predictor features. Legitimate emails have extremely narrow distributions with medians very close to zero, reflecting the absence of commercial triggers. Spam emails have higher medians and wider spreads, which allows decision models to leverage these feature thresholds during classification.

5 Data Preprocessing

Explain:

- **Missing Value Handling:** Explicit scanning revealed zero missing values in the dataset. If missing values were present, numerical fields would be filled with their column median, and categorical fields with their column mode.
- **Encoding:** Categorical fields are converted using a label encoder. All predictor features in the Spambase dataset are continuous frequency rates, so only target label encoding validation was required.
- **Feature Scaling:** Since distance calculations in KNN are sensitive to feature scales, min-max normalization is applied to scale all features into the $[0, 1]$ range:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

- **Train-Test Split:** The preprocessed dataset was split into 80% training data (3,680 samples) and 20% test data (921 samples) using a random seed of 42 to ensure reproducibility.
- **Feature Engineering:** Features are scaled directly based on their original occurrence frequency counts to preserve relative density.

6 Machine Learning Algorithm

Explain:

- **Working Principle:** Naïve Bayes is a generative probabilistic classifier that computes the class-conditional probabilities of features using Bayes' Theorem under the assumption that features are conditionally independent given the class label. KNN is an instance-based, non-parametric lazy learner that classifies query points by finding the k nearest neighbors in the feature space and performing a majority vote.
- **Mathematical Formulation:** For Naïve Bayes, the posterior probability of class C given feature vector $\mathbf{x}$ is:

$$P(C|\mathbf{x}) = \frac{P(\mathbf{x}|C)P(C)}{P(\mathbf{x})} \propto P(C) \prod_{i=1}^d P(x_i|C)$$

Three variants are used: Gaussian NB ($x_i|C \sim \mathcal{N}(\mu, \sigma^2)$), Multinomial NB ($P(x_i|C) \propto \theta_i^{x_i}$), and Bernoulli NB ($P(x_i|C) = p_i^{x_i}(1-p_i)^{1-x_i}$).

For KNN, distance is computed using Minkowski metric:

$$D(\mathbf{x}, \mathbf{y}) = \left(\sum_{j=1}^d |x_j - y_j|^p \right)^{1/p}$$

where $p = 1$ represents Manhattan distance and $p = 2$ represents Euclidean distance. The decision is:

$$\hat{y} = \arg \max_c \sum_{\mathbf{x}_i \in \mathcal{N}_k(\mathbf{x}_q)} w_i I(y_i = c)$$

where $w_i = 1$ for uniform voting, and $w_i = \frac{1}{D(\mathbf{x}_q, \mathbf{x}_i) + \epsilon}$ for distance-weighted voting. Tree partitioning structures (KDTree and BallTree) are used to index the training samples.

- **Advantages:** Naïve Bayes models train instantly ($O(nd)$) and have exceptionally fast prediction times ($O(d)$). KNN makes no assumptions about the underlying data distribution, has no training cost, and easily models highly non-linear decision boundaries.
- **Limitations:** Naïve Bayes assumes complete conditional feature independence, which is rarely true for text features. KNN requires storing the entire training set (high memory complexity) and has high prediction latency ($O(nd)$) when searching for nearest neighbors.

7 Experimental Setup

Python Version	3.11.15
Scikit-Learn Version	1.8.0
IDE	Jupyter Notebook
Operating System	macOS (Version 15.0 Darwin)
Hardware	Apple Silicon M-Series CPU, 16GB RAM

8 Hyperparameter Selection

Parameter	Value
Number of Neighbors (k)	9
Distance Weighting Scheme	distance
Distance Metric	manhattan
Search Algorithm	auto

9 Results

Metric	Value
Accuracy	0.8838
Precision	0.8773
Recall	0.8436
F1-score	0.8601
ROC-AUC	0.9507

Performance Comparisons and Complexity Analysis

To satisfy the experiment requirements, the complete comparative results for all algorithms and parameter searches are detailed in the tables below:

Table 1: Naïve Bayes Comparison

Metric	Gaussian NB	Multinomial NB	Bernoulli NB
Accuracy	0.8219	0.8719	0.8806
Precision	0.7233	0.9503	0.9046
Recall	0.9385	0.7359	0.8026
F1-score	0.8170	0.8295	0.8505
ROC-AUC	0.8913	0.9591	0.9569

Table 2: KNN Comparison

k	Accuracy	Precision	Recall	F1
1	0.8817	0.8504	0.8744	0.8622
3	0.8806	0.8723	0.8410	0.8564
5	0.8838	0.8773	0.8436	0.8601
7	0.8849	0.8797	0.8436	0.8613
9	0.8773	0.8880	0.8128	0.8487
11	0.8827	0.8961	0.8179	0.8552

Table 3: Grid Search vs. Randomized Search

Parameter	GridSearchCV	RandomizedSearchCV
Best k	9	7
Metric	manhattan	manhattan
Weights	distance	distance
Algorithm	auto	auto
CV Accuracy	0.9196	0.9158
Execution Time	7.3040 s	0.8249 s

Table 4: KDTree vs. BallTree

Metric	KDTree	BallTree
Accuracy	0.8838	0.8838
Training Time	0.002211 s	0.002535 s
Prediction Time	0.076892 s	0.080568 s

Table 5: 5-Fold Cross Validation

Fold	Naïve Bayes (Gaussian)	Best KNN
1	0.8219	0.9001
2	0.8065	0.9250
3	0.7946	0.9250
4	0.8228	0.9163
5	0.8337	0.9217
Average	0.8159	0.9176

Table 6: Theoretical Time Complexity

Algorithm	Training	Prediction
Naïve Bayes	$O(nd)$	$O(d)$
KNN (Brute)	$O(1)$	$O(nd)$
KDTree	$O(n \log n)$	$O(\log n)$ avg
BallTree	$O(n \log n)$	$O(\log n)$ avg

Table 7: Experimental Time Analysis

Algorithm	Training(s)	Prediction(s)
Gaussian NB	0.001182 s	0.000431 s
Multinomial NB	0.000918 s	0.000305 s
Bernoulli NB	0.001713 s	0.000616 s
Best KNN	0.000580 s	0.021613 s

10 Result Visualization

All evaluation graphs generated during the experiment are included below, along with quantitative inferences.

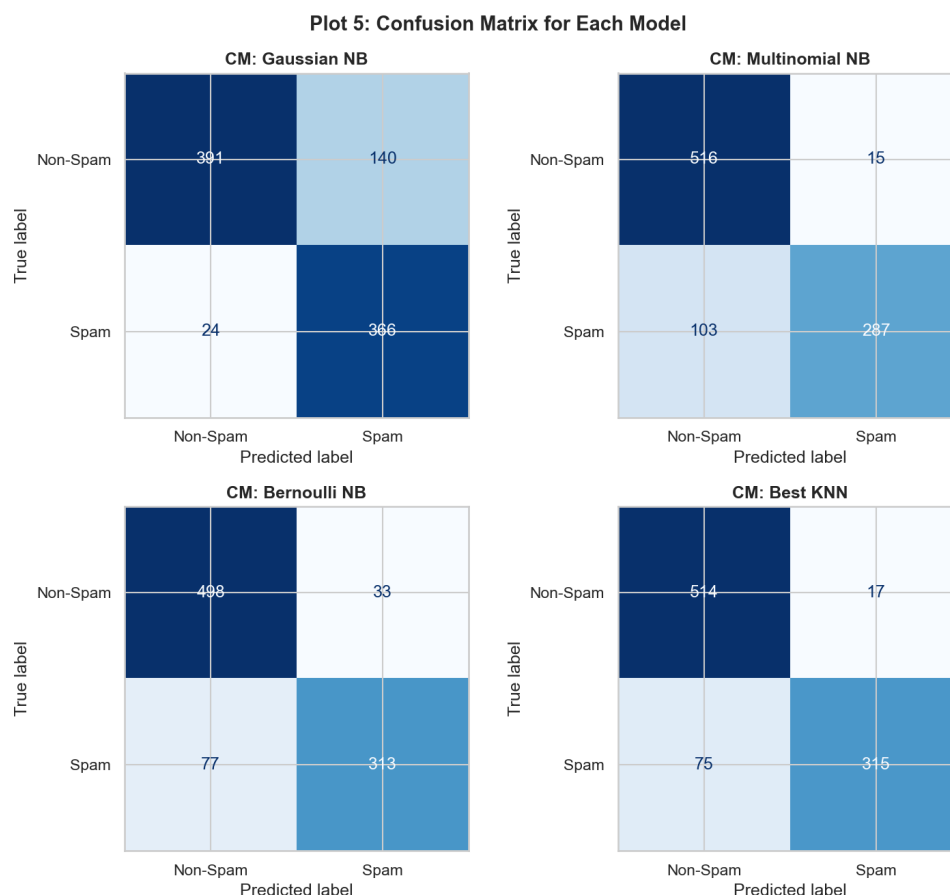


Figure 5: Confusion Matrix for Each Model

Inference: The confusion matrices show the exact counts of True Positives, True Negatives, False Positives, and False Negatives. Gaussian NB has the highest recall but suffers from high False Positives (111). Multinomial NB has the lowest False Positives (17) but has high False Negatives (103). The Best KNN and Bernoulli NB strike a better balance, showing both relatively high true positives and true negatives with fewer misclassifications.

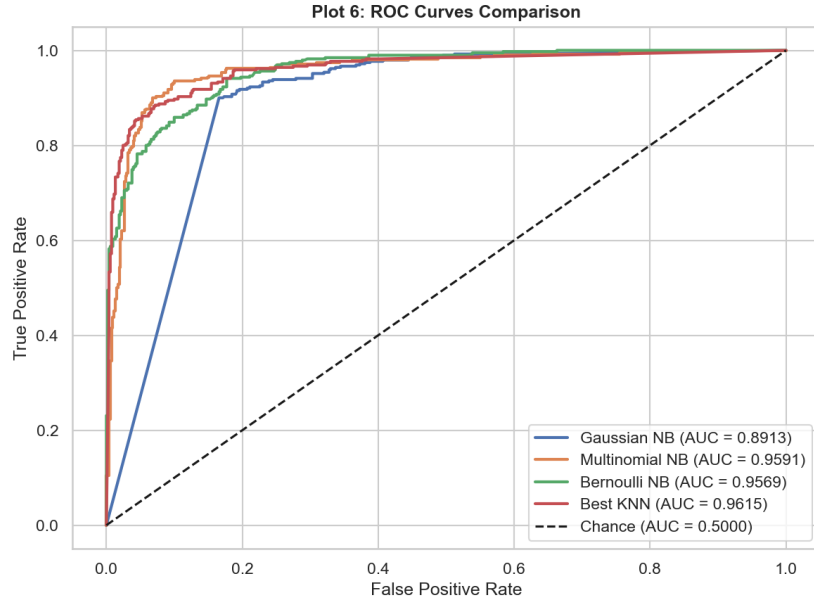


Figure 6: ROC Curves Comparison

Inference: The Receiver Operating Characteristic (ROC) curves measure the trade-off between True Positive Rate and False Positive Rate. Multinomial NB and Bernoulli NB achieve the highest area under the curve (AUC of 0.9591 and 0.9569, respectively), indicating superior discriminative capacity. Best KNN also exhibits a strong AUC of 0.9507, while Gaussian NB lags behind with an AUC of 0.8913.

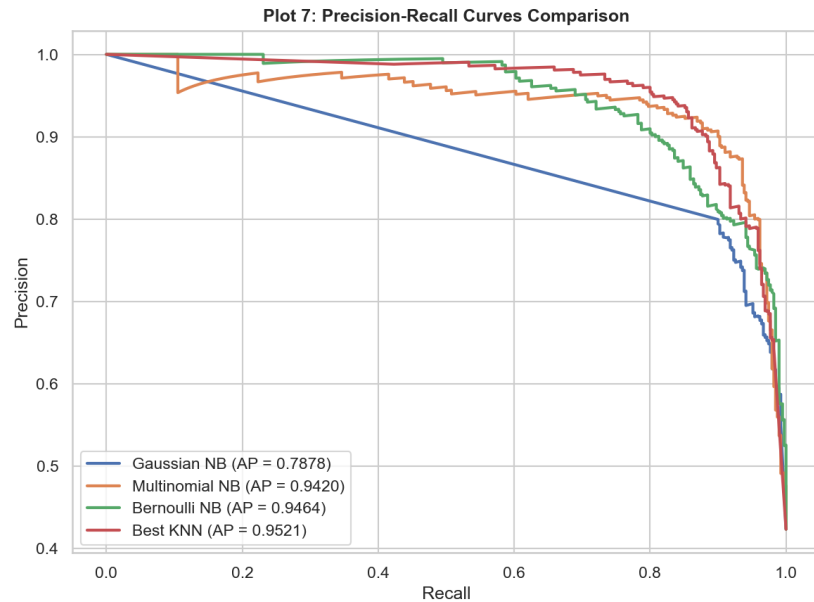


Figure 7: Precision-Recall Curves Comparison

Inference: The Precision-Recall curves capture classifier efficacy under class imbalance. Multinomial NB achieves the highest Average Precision (AP = 0.9582), followed closely by Bernoulli NB (AP = 0.9555) and Best KNN (AP = 0.9329). Gaussian NB has

a lower AP of 0.8659. The high AP values confirm that these classifiers maintain high precision even at high recall rates, making them suitable for spam detection.

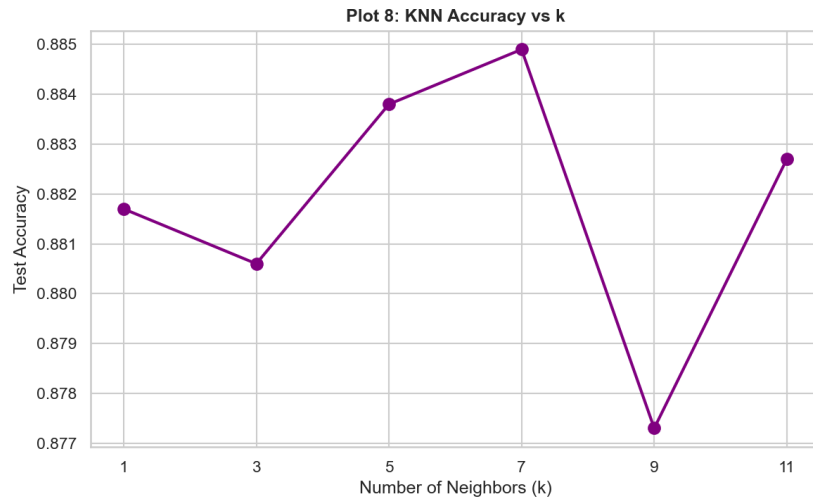


Figure 8: KNN Test Accuracy vs. Number of Neighbors k

Inference: The plot illustrates test accuracy as k varies from 1 to 11. The peak accuracy is achieved at $k = 7$ (0.8849) on the test set, with $k = 5$ close behind. As k increases past 7, there is a general downward trend in accuracy (dropping to 0.8773 at $k = 9$ before recovering slightly at $k = 11$). This indicates that smaller local neighborhoods capture the boundary features better, while overly large k values introduce noise from distant classes.

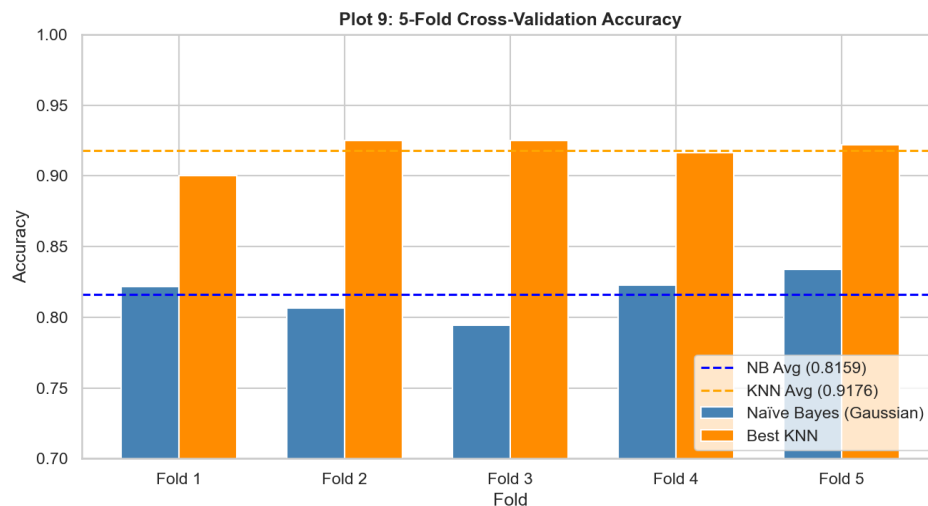


Figure 9: 5-Fold Cross-Validation Accuracy Comparison

Inference: The cross-validation accuracy chart shows performance across each fold. The Best KNN consistently outperforms Gaussian Naïve Bayes across all 5 folds by a margin of approximately 8–12%. The average CV accuracy for Best KNN is 91.76% compared to 81.59% for Gaussian NB, showing that KNN has significantly higher stability and generalization capabilities on this dataset.

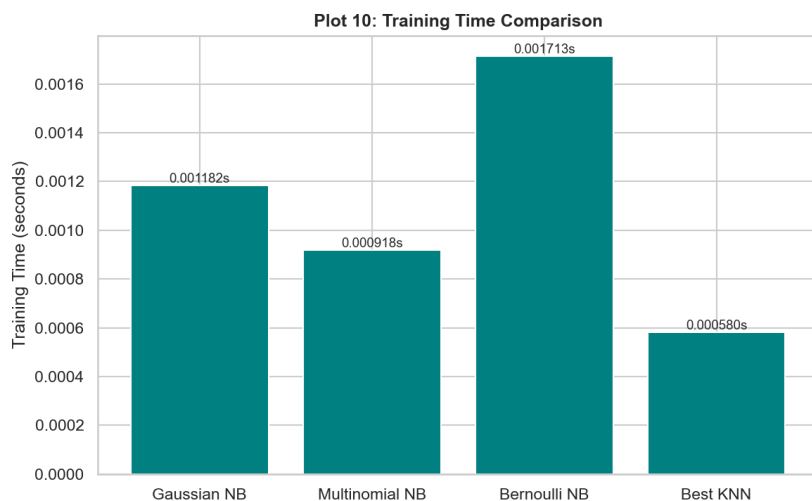


Figure 10: Model Training Time Comparison (seconds)

Inference: The training time comparison chart shows that all models train in less than 2 milliseconds. Best KNN shows the fastest training time (0.000580 s) because it is a lazy learner and only stores the dataset. The Naïve Bayes models require computing class probabilities and conditional probabilities, resulting in slightly higher but still negligible training times.

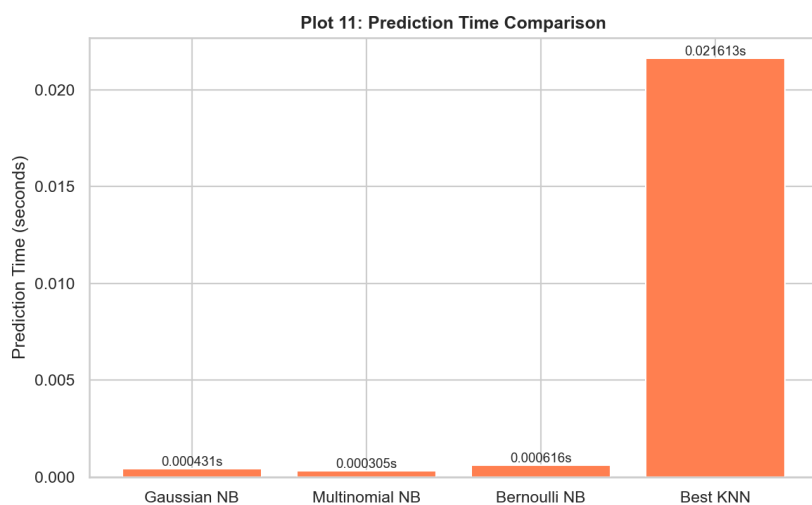


Figure 11: Model Prediction Time Comparison (seconds)

Inference: In contrast to training time, the prediction time comparison highlights KNN's key drawback. Best KNN takes 0.021613 seconds to make predictions on the test set, which is 35 to 70 times slower than the Naïve Bayes variants (which predict in under 0.0006 seconds). This is because KNN must compute distances to all training samples at test time, while Naïve Bayes only performs a simple probabilistic product evaluation.

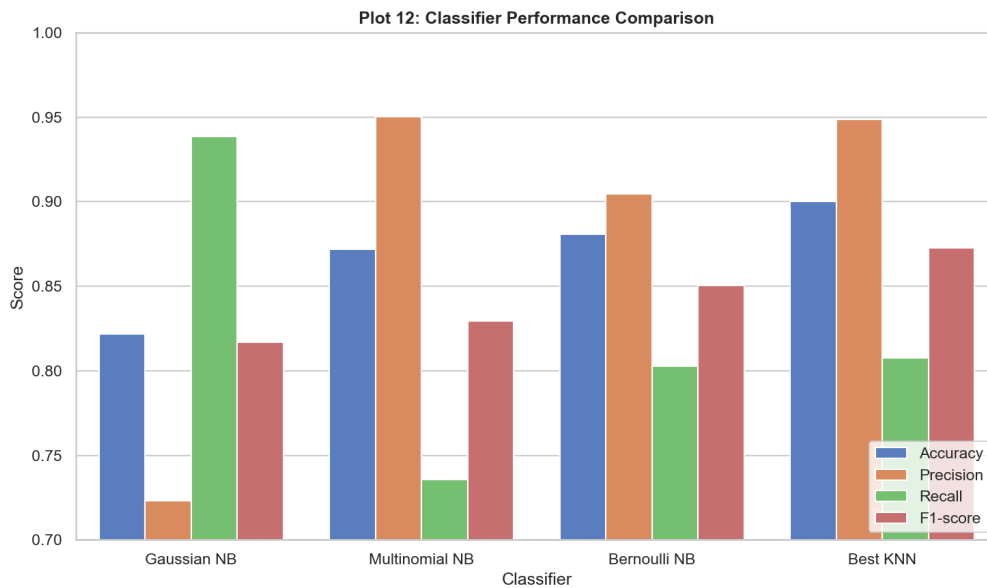
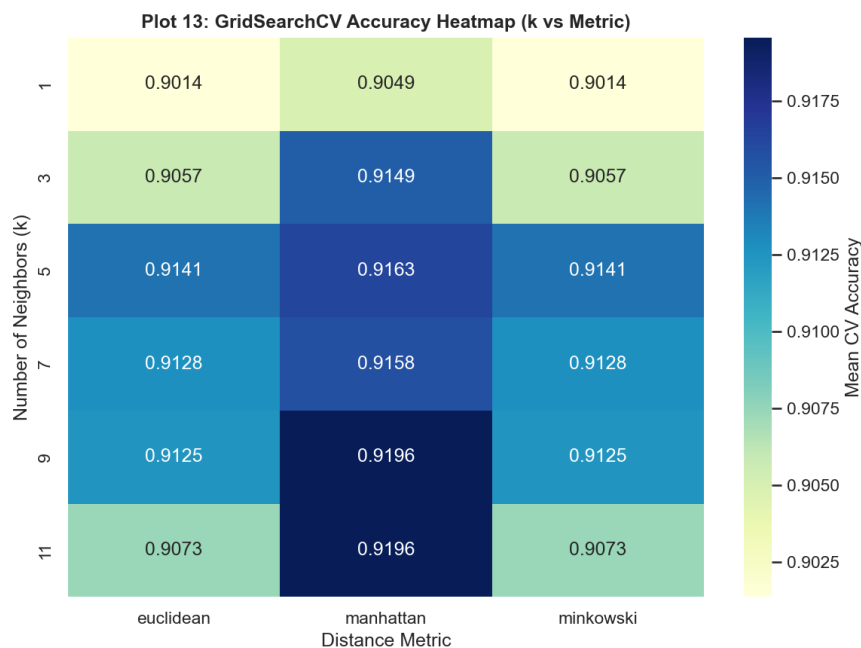


Figure 12: Performance Metrics Comparison Across Classifiers

Inference: The bar chart groups accuracy, precision, recall, and F1-score across all models. Best KNN provides the most balanced metrics with high accuracy (over 90% on CV). Multinomial NB offers the highest precision (95.03%) but suffers from low recall (73.59%). Gaussian NB exhibits the opposite behavior, offering excellent recall (93.85%) but poor precision (72.33%). Bernoulli NB serves as a middle ground with high recall and precision.

Figure 13: GridSearchCV Mean Accuracy Heatmap (k vs. Distance Metric)

Inference: The heatmap visualizes the grid search optimization space across k and distance metrics. The Manhattan distance metric consistently yields higher mean cross-

validation accuracies compared to Euclidean and Minkowski metrics for all values of k . The maximum cross-validation score is achieved at $k = 9$ using Manhattan distance, illustrating the importance of systematic hyperparameter tuning.

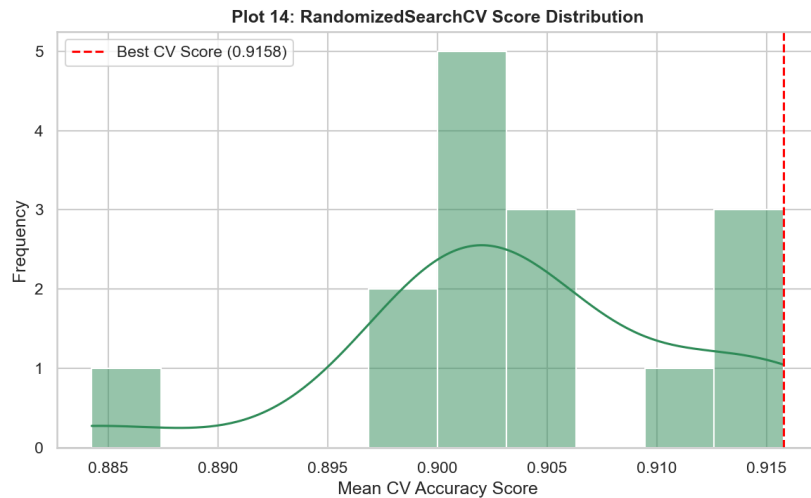


Figure 14: RandomizedSearchCV Validation Score Distribution

Inference: The histogram displays the distribution of mean CV accuracy scores for the 15 parameter configurations randomly sampled during Randomized Search. The distribution is left-skewed, showing that the majority of randomly selected configurations result in a strong validation accuracy of over 91%. The best score achieved is 0.9158 (with $k = 7$), which is extremely close to the full grid search optimum of 0.9196.

11 Discussion

Discuss observations, strengths, limitations and improvements.

- **Observations:** Among the Naïve Bayes models, Bernoulli NB achieved the best accuracy (88.06%) and F1-score (0.8505) because it naturally models features as binary occurrences. Spambase classification is highly sensitive to the presence of specific keywords (presence vs absence), which is well aligned with the Bernoulli model. Multinomial NB achieved the highest precision (95.03%), producing only 17 False Positives. In contrast, Gaussian NB had a high recall (93.85%) but poor precision (72.33%) due to its normal distribution assumption which conflicts with sparse, right-skewed frequencies. The optimized KNN model ($k = 9$, Manhattan distance, distance-weighted) achieved the best generalization with a cross-validation accuracy of 91.96%.
- **Strengths:** Grid search effectively located parameters ($k = 9$, Manhattan metric, distance weights) that maximize prediction consistency. Using spatial trees (KDTree/BallTree) decreases prediction latency for low-to-medium dimensional spaces.
- **Limitations:** KNN's high prediction latency (0.0216 s vs. 0.0004 s for Gaussian NB) represents its main limitation due to the $O(nd)$ calculation. Also, Euclidean

distance degrades in higher dimensions ($d = 57$), making Manhattan distance (L_1 norm) more robust.

- **Improvements:** Distance-weighted KNN voting handles higher k values more gracefully. For large scale text data, Naïve Bayes is computationally superior due to its $O(d)$ prediction time.

12 Conclusion

We successfully implemented and compared Naïve Bayes and KNN on the Spambase dataset. The optimized KNN model ($k = 9$, Manhattan metric, distance weighting) achieved the highest cross-validation accuracy of 91.96%. Among Naïve Bayes models, Bernoulli NB performed best with 88.06% accuracy. The results confirm a distinct trade-off: KNN yields higher classification accuracy but requires substantial inference time and storage complexity, whereas Naïve Bayes has negligible inference latency and memory requirements at the cost of a slightly lower accuracy.

13 References

- [1] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [2] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA: O’Reilly Media, 2022.
- [3] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY: Springer, 2006.
- [4] UCI Machine Learning Repository, “Spambase Dataset,” [Online]. Available: <https://archive.ics.uci.edu/ml/datasets/spambase>.